Otimização de Minimização de Custo da Nutrição — Faz. Campo Dourado (GO)

> Dada a **Fazenda GO_campo_dourado** com **clima de Goiás**, para fazermos a otimização da **minimização de custo com nutrição** será necessário avaliar os **alimentos** (matéria seca, matéria molhada/natural, etc.) e respeitar as **restrições nutricionais**, uma vez que elas já estão estabelecidas no padrão **NASEM / NRC** (nutrient_standard).

Este documento é um guia prático de como montar o problema de **Programação Linear (PL)** para minimizar o custo da dieta no RuFaS, explicando:

1. Contexto e dados reais da fazenda campo_dourado
2. Os **parâmetros de entrada** que você deve alterar para otimizar
3. A **Função Objetivo**
4. As **Restrições** (padrão NASEM)
5. Um **modelo pronto para o Solver** (Excel/LibreOffice)
6. O **método Simplex** (passo a passo e observações)
7. O plano de **análise de sensibilidade** variando as simulações após o ótimo

---

1. Contexto da fazenda

Dados reais extraídos de `output/synthetic_farms/campanha_80_fazendas/GO_campo_dourado/`:

Item	Valor
Nome	Faz. Campo Dourado (Goiás)
Slug	`GO_campo_dourado`
Estado / Clima	Goiás — `input/data/weather/BR_GO_weather.csv`
Cenário	conservador (100 vacas, foco em estabilidade)
Padrão nutricional	**NASEM** (`config.nutrient_standard`)
Rebanho	`cow_num = 100`, `milking_cow_fraction = 0.8356`
Ordenhas/dia	`cow_times_milked_per_day = 3`
Intervalo de reformulação	`ration.formulation_interval = 45` dias
Buffer de fósforo	`ration.phosphorus_requirement_buffer = 50` (= 150% do requisito calculado)
Mitigação de metano	`3-NOP` (`3-NOP_additive_amount = 90`)
Período de simulação	2015:1 → 2019:365
Arquivo de alimentação	`input/data/feed/example_Northeast_feed.json`

---

2. Alimentos disponíveis (os "x" do problema)

No RuFaS cada alimento é identificado por um `feed_type`, que corresponde ao `rufas_id` da tabela `user_feeds.csv`. A composição nutricional vem de

`NASEM_Comp.csv` (base úmida/natural na coluna `DM` = % de matéria seca).

Alimentos efetivamente usados nas rações da GO_campo_dourado:

feed_type	Alimento	Categoria	DM (%MS)	CP (%)	NDF (%)	Ca (%)	P (%)	custo (`purchased_feed_cost`)
18	Bermudagrass, feno	Forragem	93.37	10.99	65.41	0.195	0.208	0.01
23	Blood meal, low dRUP	Concentrado proteico	90.86	95.06	0.00	0.276	0.048	0.10
44	Milho grão moído seco	Energético	86.91	8.51	9.76	0.309	0.132	0.50
51	Silagem de milho, meia maturação	Forragem	—	—	—	—	—	0.01
94	Mistura capim-leguminosa, feno	Forragem	—	—	—	—	—	0.01
100	Mistura capim-leguminosa, silagem	Forragem	—	—	—	—	—	0.01
110	Leguminosa forrageira, silagem	Forragem	—	—	—	—	—	0.01
176	Sudan grass, silagem madura	Forragem	—	—	—	—	—	0.10
202	Leite integral	Leite (bezerros)	—	—	—	—	—	0.10
216	Starter 18CP high starch (bezerros)	Starter	—	—	—	—	—	0.10
301	Premix mineral/vitamínico	Mineral	—	—	—	—	—	1.00
303	Premix mineral/vitamínico	Mineral	—	—	—	—	—	0.10

> Os valores em branco devem ser preenchidos a partir de `NASEM_Comp.csv` (localize a linha pelo nome do alimento em `user_feeds.csv`). O custo vem do campo `purchased_feed_cost` dentro de `feed.json`.

Matéria seca (MS) × matéria natural/molhada (MN)

- A tabela `NASEM_Comp.csv` traz os nutrientes **em base de matéria seca** e a coluna `DM` indica o **percentual de MS** de cada alimento.
- A relação é: `MN = MS / (DM/100)`.
  - Forragens úmidas (silagens) têm DM baixo (ex.: silagem de cevada ~37% → muita água).
  - Concentrados/fenos têm DM alto (~86–93%).
- **Formule a dieta sempre em base de matéria seca** (os requisitos NASEM são em MS) e só depois converta para matéria natural para saber quanto comprar/pesar no cocho.

---

3. Parâmetros de entrada que você deve alterar para otimizar

Você vai mexer principalmente em `feed.json` (alimentos e rações) e em alguns parâmetros de `ration` / `config`.

3.1 Em `feed.json` → seção `feeds` (candidatos e custos)

Campo	O que é	Como usar na otimização
`feed_type`	ID do alimento candidato	Liste **sem repetir** todos os alimentos que o Solver pode escolher
`purchased_feed_cost`	**Custo por kg**	É o **coeficiente da função objetivo** (c_j)

Campo	O que é	Como usar na otimização
`buffer`	Folga de compra	Mantenha padrão (0.15) salvo necessidade
`runtime_purchase_allowance` / `advance_purchase_allowance` / `planning_cycle_allowance`	Limites de compra	Vira **restrição de disponibilidade** (limite superior de x_j)

3.2 Em `feed.json` → seção `rations` (a dieta ótima)

Campo	O que é
`animal_combination`	Categoria: `calf`, `growing`, `close_up`, `lac_cow`
`intake_option`	`set_DMI` (fixa o consumo) ou `predict_DMI`
`intake_value`	Consumo de MS (kg/dia) quando `set_DMI`
`feeds[].ration_percentage`	**% de cada alimento na dieta** → é a variável de decisão x_j que o Solver vai otimizar; a soma deve dar 100

3.3 Parâmetros de `ration` / `config`

Caminho	Padrão	Papel na otimização
`config.nutrient_standard`	`NASEM`	Define a tabela de composição e os requisitos (NASEM ou NRC)
`ration.formulation_interval`	45	Com que frequência reformula (impacta sensibilidade sazonal)
`ration.phosphorus_requirement_buffer`	50	Aperta/afrouxa a restrição de P (50 = exigir 150% do P)
`ration.user_input`	True	`True` = você fornece a dieta otimizada manualmente
`ration.maximum_ration_reformulation_attempts`	250	Teto de tentativas (deixe no padrão)

Caminho	Padrão	Papel na otimização
`ration_formulation_parameters.user_defined_ration_tolerance`	0.1	Tolerância da dieta fornecida pelo usuário

> **Fluxo recomendado**: resolva a PL por fora (Solver/Simplex) para achar as `ration_percentage` ótimas de cada categoria, coloque-as em `feed.json` com `ration.user_input = True` e rode a simulação para validar.

---

4. Função Objetivo

Minimizar o **custo total da dieta** (por kg de matéria seca, ou por dia por animal).

Seja `x_j` a quantidade (kg de MS) do alimento `j` na dieta e `c_j` o custo por kg (`purchased_feed_cost`):

```
Minimizar  Z = Σ_j ( c_j · x_j )
```

Se preferir trabalhar com **proporções** `p_j` (as `ration_percentage`, somando 1):

```
Minimizar  Z = Σ_j ( c_j · p_j )      sujeito a   Σ_j p_j = 1
```

- Variáveis de decisão: `x_j ≥ 0` (kg de MS de cada alimento) — ou `p_j` em %.
- Coeficientes `c_j`: coluna `purchased_feed_cost` do `feed.json`.

---

5. Restrições (padrão NASEM)

As restrições vêm dos **requisitos nutricionais NASEM** para a categoria animal (ex.: `lac_cow`). Todas em **base de matéria seca**. Monte uma restrição por nutriente. Formato geral:

```
Σ_j ( a_ij · x_j )  (≥ | ≤ | =)  b_i
```

onde `a_ij` é o teor do nutriente `i` no alimento `j` (de `NASEM_Comp.csv`) e `b_i` é o requisito/limite NASEM.

5.1 Restrição de quantidade total (DMI)

```
Σ_j x_j = DMI        (consumo total de matéria seca, kg/dia)
```

Para vaca em lactação, use o DMI previsto pelo NASEM ou o `intake_value` definido.

5.2 Restrições de energia e proteína (mínimos, "≥")

Nutriente	Coluna em NASEM_Comp.csv	Tipo	Requisito b_i
Energia (NEl / DE)	`DE` / `TDN`	≥	mínimo NASEM da categoria
Proteína bruta (CP)	`CP`	≥	mínimo NASEM
Proteína não degradável (RUP)	`RUP`	≥	mínimo NASEM

5.3 Restrições de fibra (faixa, "≥" e "≤")

Nutriente	Coluna	Tipo	Observação
FDN (NDF)	`NDF`	≥	mínimo para saúde ruminal
FDN forragem	`NDF` (só forragens)	≥	fração mínima de fibra de forragem
Amido (starch)	`starch`	≤	teto para evitar acidose

5.4 Restrições de minerais

Mineral	Coluna	Tipo	Requisito
Cálcio (Ca)	`calcium`	≥ (e ≤)	requisito NASEM, respeitar relação Ca:P
Fósforo (P)	`phosphorus`	≥	requisito × (1 + phosphorus_requirement_buffer/100) = ×1,5
Magnésio, Potássio, Sódio, Enxofre	`magnesium`, `potassium`, `sodium`, `sulfur`	≥	requisitos NASEM

5.5 Restrições estruturais

```
Σ_j p_j = 100 %            (fecha a dieta)
x_j ≥ 0        ∀ j         (não-negatividade)
x_j ≤ U_j                  (limites de inclusão / disponibilidade, ex.: *_purchase_allowance)
```

Opcional: limites mínimos/máximos de inclusão por alimento (ex.: silagem de milho entre 20% e 60% da MS) para evitar soluções irrealistas.

> **Onde achar os b_i**: os requisitos NASEM para cada categoria/estágio são calculados pelo próprio RuFaS. Você pode extraí-los dos relatórios da simulação

> (variáveis de requisito nutricional reportadas) e usá-los como os lados direitos `b_i` da PL.

---

6. Modelo para o Solver (Excel / LibreOffice Calc)

Monte uma planilha assim (exemplo para `lac_cow`):

Layout das células

	A	B	C	D	E	F
1	Alimento (feed_type)	**x_j (kg MS)**	Custo c_j	CP %	NDF %	P %
2	44 — Milho moído	`célula var.`	0.50	8.51	9.76	0.132
3	51 — Silagem milho	`célula var.`	0.01	...	...	...
4	110 — Silagem legum.	`célula var.`	0.01	...	...	...
5	303 — Premix mineral	`célula var.`	0.10	...	...	...
...	...	...	...	...	...	...

- **Célula objetivo (custo total)**: `=SOMARPRODUTO(B2:B10; C2:C10)` → minimizar.
- **Células variáveis**: `B2:B10` (os x_j).
- **Restrições** (adicionar no diálogo do Solver):
  - `SOMA(B2:B10) = DMI` (quantidade total de MS)
  - `SOMARPRODUTO(B2:B10; D2:D10) >= CP_min` (proteína)
  - `SOMARPRODUTO(B2:B10; E2:E10) >= NDF_min` (fibra)

- `SOMARPRODUTO(B2:B10; F2:F10) >= P_req × 1,5` (fósforo com buffer de 50%)
- demais nutrientes de forma análoga
- `B2:B10 >= 0`

Configuração do Solver

- Objetivo: célula do custo total → **Mín**.
- Método: **LP Simplex** (Simplex LP no Excel; "Programação linear" no LibreOffice).
- Marque "Tornar variáveis irrestritas não negativas".

Resultado: os `x_j` ótimos. Converta para `ration_percentage` com `p_j = 100 · x_j / Σ x_j` e cole em `feed.json`.

Equivalente em Python (opcional, com PuLP)

```
import pulp

# custos (purchased_feed_cost) e teores (de NASEM_Comp.csv, base MS)
feeds   = ["milho44", "sil_milho51", "sil_leg110", "premix303"]
custo   = {"milho44": 0.50, "sil_milho51": 0.01, "sil_leg110": 0.01, "premix303": 0.10}
cp      = {"milho44": 8.51, "sil_milho51": 7.0,  "sil_leg110": 15.0, "premix303": 0.0}
ndf     = {"milho44": 9.76, "sil_milho51": 45.0, "sil_leg110": 42.0, "premix303": 0.0}
p_teor  = {"milho44": 0.132,"sil_milho51": 0.25, "sil_leg110": 0.30, "premix303": 20.0}

DMI, CP_min, NDF_min, P_req = 23.0, 16.0, 28.0, 0.35  # exemplos — use os requisitos NASEM reais

prob = pulp.LpProblem("dieta_min_custo", pulp.LpMinimize)
x = {f: pulp.LpVariable(f, lowBound=0) for f in feeds}

prob += pulp.lpSum(custo[f] * x[f] for f in feeds)          # função objetivo
prob += pulp.lpSum(x[f] for f in feeds) == DMI              # DMI total
prob += pulp.lpSum(cp[f]  * x[f] for f in feeds) >= CP_min  * DMI / 100
```

```
prob += pulp.lpSum(ndf[f] * x[f] for f in feeds) >= NDF_min * DMI / 100
prob += pulp.lpSum(p_teor[f] * x[f] for f in feeds) >= P_req * 1.5  # buffer de P = 50%

prob.solve()
for f in feeds:
    print(f, round(x[f].value(), 3), "kg MS")
print("Custo diário:", round(pulp.value(prob.objective), 4))
```

---

7. Método Simplex

O Simplex é o algoritmo clássico para resolver PL. Como aplicá-lo aqui:

1. **Forma padrão**: escreva a função objetivo e as restrições; transforme desigualdades em igualdades com variáveis de folga (`≤`) e de excesso + artificiais (`≥`).
   - `Σ a_ij x_j ≥ b_i` → `Σ a_ij x_j - s_i + A_i = b_i`
   - `Σ a_ij x_j ≤ b_i` → `Σ a_ij x_j + s_i = b_i`
2. **Quadro inicial (tableau)**: monte a matriz com os coeficientes e a linha Z. Para `≥` e `=`, use o método das **duas fases** ou do **Big-M** (por causa das variáveis artificiais).
3. **Iterações**:
   - Escolha a coluna pivô (variável que entra): a de maior melhora em Z (minimização → coeficiente reduzido mais negativo).
   - Escolha a linha pivô (variável que sai): menor razão `b_i / a_i,pivô` positiva.
   - Pivoteie (normalize a linha pivô e zere a coluna nas demais).
4. **Parada**: quando nenhum coeficiente reduzido indicar melhora → solução ótima.

5. **Leitura**: as variáveis básicas dão os `x_j` ótimos; a linha Z dá o custo mínimo; os preços-sombra (dualidade) saem da linha Z nas colunas de folga.

> Para dietas, o número de nutrientes (restrições) costuma ser maior que o de alimentos — o Simplex lida bem com isso. Na prática use o Simplex LP do Solver; faça o quadro à mão só para o entendimento didático que o professor pediu.

---

8. Análise de sensibilidade (variando as simulações após o ótimo)

Depois de achar o ótimo (dieta de custo mínimo que atende NASEM), varie **um parâmetro por vez** e registre o impacto no custo e na composição.

8.1 O que o próprio Simplex já entrega

- **Preços-sombra** (duais): quanto o custo muda se afrouxar/apertar 1 unidade de cada restrição (ex.: valor marginal de +1 ponto de PB exigida).
- **Faixas de sensibilidade dos custos (c_j)**: quanto o `purchased_feed_cost` de cada alimento pode variar **sem mudar a dieta ótima**.
- **Faixas dos lados direitos (b_i)**: intervalo em que o preço-sombra permanece válido.

No Solver, marque **"Relatório de Sensibilidade"** ao resolver.

8.2 Cenários a simular no RuFaS (um fator por vez)

#	Parâmetro variado	Valores sugeridos	O que observar
1	`purchased_feed_cost` do milho (44)	0.3 / 0.5 / 0.8 / 1.2	ponto em que o milho sai/entra na dieta

#	Parâmetro variado	Valores sugeridos	O que observar
2	`purchased_feed_cost` das silagens	±25%	troca entre forragens
3	`ration.phosphorus_requirement_buffer`	0 / 25 / 50 / 75	custo extra do P e inclusão de premix mineral
4	`config.nutrient_standard`	NASEM vs NRC	mudança de requisitos e composição
5	`ration.formulation_interval`	15 / 30 / 45 / 60	efeito sazonal (clima GO) no custo médio
6	DMI / nível de produção de leite	±10%	elasticidade do custo com a produção
7	Disponibilidade (`*_purchase_allowance`)	apertar limites	restrição ativa e novo ótimo

8.3 Procedimento recomendado

1. Fixe o caso base = **dieta ótima** encontrada.
2. Para cada cenário, altere **só um** parâmetro, rode a simulação RuFaS (`main.py`) e extraia o custo da dieta e as `ration_percentage` resultantes.
3. Monte uma tabela/gráfico **custo × parâmetro** (curva de sensibilidade).
4. Identifique os **pontos de quebra** (onde a base ótima muda) — eles batem com as faixas de sensibilidade que o Simplex previu.

> Dica: como são 5 fazendas de Goiás (seeds 42–46), você pode repetir a análise em `GO_serra_do_cerrado` , `GO_boa_vereda` , `GO_rio_vermelho` e `GO_chapada_verde` para comparar a robustez do ótimo entre fazendas do mesmo clima.

---

9. Checklist de execução

- [ ] Preencher a tabela de composição (seção 2) a partir de `NASEM_Comp.csv`.
- [ ] Levantar os requisitos NASEM (b_i) por categoria a partir dos relatórios RuFaS.
- [ ] Montar a PL (função objetivo + restrições) por categoria animal.
- [ ] Resolver com Solver (Simplex LP) e gerar o Relatório de Sensibilidade.
- [ ] Converter x_j ótimos em `ration_percentage` e atualizar `feed.json`.
- [ ] Rodar a simulação com `ration.user_input = True` para validar.
- [ ] Executar os 7 cenários de sensibilidade e tabular custo × parâmetro.

Arquivos de referência (fonte dos dados)

- `output/synthetic_farms/campanha_80_fazendas/GO_campo_dourado/parametrizacao.md`
- `output/synthetic_farms/campanha_80_fazendas/GO_campo_dourado/inputs_resolvidos/feed.json`
- `output/synthetic_farms/campanha_80_fazendas/GO_campo_dourado/inputs_resolvidos/NASEM_Comp.csv`
- `output/synthetic_farms/campanha_80_fazendas/GO_campo_dourado/inputs_resolvidos/user_feeds.csv`
- `output/synthetic_farms/campanha_80_fazendas/GO_campo_dourado/inputs_resolvidos/economy.csv`
- `Documentação/ObsidianVariaveis/Racao/` (notas de parâmetros de ração)